Unleash migration checklist

Use this checklist to plan and track your migration to Unleash. It follows the five phases of the migration guide: plan, pilot, migrate, cut over, and onboard. Phases 1, 2, 4, and 5 are done once for the whole migration; phase 3 repeats for every team or application you migrate.

Phase 1: Plan

Scope

Depending on your organization, some of this work, such as inventorying and classifying flags, may be owned by individual teams rather than done centrally. Decide which model fits you, and move team-owned items into the per-team checklist in Phase 3.

- [ ] Export a full inventory of flags from the current system (name, owner, created date, last evaluated)
- [ ] Classify every flag as active, permanent, or stale (see flag lifecycle)
- [ ] Approve a deletion list for stale flags; remove them from code and delete them
- [ ] Confirm the two-track approach: all new flags are created in Unleash from day one, and legacy flags are migrated separately
- [ ] If possible, disable the creation of new flags in the old system to enforce the two-track approach
- [ ] List all codebases and services that evaluate flags, with their languages and frameworks (see Unleash SDKs)
- [ ] List all downstream consumers of flag data, for example Jira, Datadog, Slack, or analytics (see integrations)

Business case and stakeholders

- [ ] Document the pain the migration solves: deployment, debugging, and rollback practices that slow teams down today
- [ ] Document what "better" looks like after the migration, with measurable outcomes
- [ ] Identify the platform owner who will administer Unleash
- [ ] Identify the day-to-day users: which teams, how many developers, which non-engineering roles
- [ ] Identify sign-off owners: security, change management, procurement, legal
- [ ] Confirm timeline expectations and budget with the sign-off owners

- [ ] Agree on success metrics, for example teams onboarded, legacy flags migrated or deleted, and old-system evaluation traffic

Target state

- [ ] Choose the organizing principle for projects: by team, application, or business domain (see organizing feature flags)
- [ ] Define the environment setup and map each old environment to an Unleash environment
- [ ] Define flag naming conventions and configure naming patterns per project
- [ ] Map current roles and permissions to Unleash predefined or custom roles
- [ ] Decide on hosting: Unleash-hosted, hybrid, or self-hosted
- [ ] Decide whether Unleash Edge is needed for frontend traffic or scale
- [ ] Review security and compliance requirements, for example SOC 2, ISO 27001, GDPR, or HIPAA
- [ ] Plan single sign-on and SCIM provisioning
- [ ] Review Unleash resource limits against your expected flag volume

Instance setup

- [ ] Provision the Unleash instance and pass internal security review
- [ ] Codify projects, environments, roles, API tokens, and SSO with the Terraform provider, or configure them in the Admin UI
- [ ] Create custom context fields for every user or request attribute used in targeting
- [ ] Create shared segments
- [ ] Create service accounts and scoped API tokens for migration scripts
- [ ] Set up integrations: alerting, ticketing, chat, and analytics
- [ ] Set up the Unleash MCP server and AI coding assistant access, if used for the migration

Phase 2: Pilot

- [ ] Select one or two pilot teams and confirm their use cases match the legacy platform's
- [ ] Wrap flag evaluation in an abstraction layer, or adopt OpenFeature, in the pilot codebases
- [ ] Run both systems in parallel and log evaluation mismatches
- [ ] Resolve all mismatches and document their causes for later teams
- [ ] Collect pilot feedback and update onboarding material
- [ ] Get sign-off to roll out beyond the pilot

Phase 3: Migrate

Copy this section for each team or application.

Team or application: _____ Owner: _____ Target date: _____

- [ ] Review the team's flag list; delete stale flags instead of migrating them
- [ ] Create required context fields and segments in Unleash
- [ ] Recreate the team's flags and targeting rules, manually or scripted through the Admin API
- [ ] Review every translated targeting rule against the source system
- [ ] Update application code to evaluate flags through the abstraction layer or Unleash SDKs
- [ ] Verify configuration in a non-production environment
- [ ] Run in parallel in production and monitor the mismatch rate
- [ ] Handle partially rolled-out percentage flags explicitly, since rollout buckets change between systems (see stickiness)
- [ ] Switch the team's flags to Unleash once the mismatch rate reaches zero
- [ ] Remove old SDK calls and dependencies from the team's codebases
- [ ] Give the team access and permissions in Unleash

Phase 4: Cut over

- [ ] Disable the creation of new flags in the old system, if not already done at the start of the migration
- [ ] Monitor the old system for remaining evaluation traffic
- [ ] Remove the old SDKs and dependencies from all codebases
- [ ] Archive or export the old system's audit history if required by compliance
- [ ] Decommission the old system once evaluation traffic reaches zero
- [ ] Cancel licenses or shut down infrastructure for the old system
- [ ] Hold a retrospective and record lessons learned

Phase 5: Onboard

- [ ] Enable single sign-on and SCIM provisioning for all users
- [ ] Grant project permissions per team
- [ ] Deliver training, with Unleash Customer Success or self-paced through the Unleash Learning Lab
- [ ] Publish internal documentation: naming conventions, flag lifecycle policy, and support channels

- [ ] Set up flag hygiene routines: lifecycle tracking, health metrics, and cleanup cadence